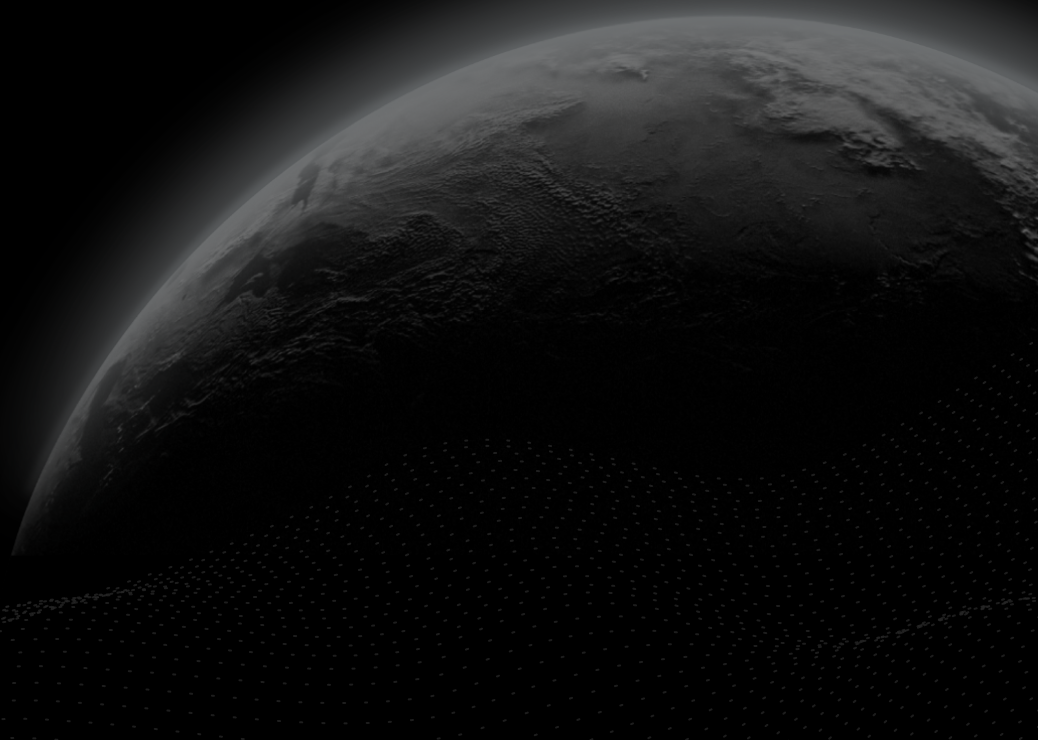
CERTIK

Security Assessment

Brickken - Audit

CertiK Assessed on Dec 23rd, 2022



CertiK Assessed on Dec 23rd, 2022

Brickken - Audit

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES	ECOSYSTEM	METHODS
DeFi	Ethereum (ETH)	Manual Review, Static Analysis

LANGUAGE	TIMELINE	KEY COMPONENTS
Solidity	Delivered on 12/23/2022	N/A

CODEBASE
https://github.com/Brickken/brickken-protocol
View All in Codebase Page

COMMITS
c575b55d3c39a156700b05ac5218d6e23f79ca15
View All in Codebase Page

Vulnerability Summary

15	11	0	0	4	0
Total Findings	Resolved	Mitigated	Partially Resolved	Acknowledged	Declined

1	Critical	1 Resolved	Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
3	Major	3 Acknowledged	Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
2	Medium	2 Resolved	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
4	Minor	3 Resolved, 1 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
5	Informational	5 Resolved	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | BRICKKEN - AUDIT

CODEBASE | BRICKKEN - AUDIT

Repository

https://github.com/Brickken/brickken-protocol

Commit

c575b55d3c39a156700b05ac5218d6e23f79ca15

AUDIT SCOPE | BRICKKEN - AUDIT

13 files audited ● 8 files with Acknowledged findings ● 5 files with Resolved findings

ID	File	SHA256 Checksum
● UBU	sto/UpgradeableBeacon/UpgradeableBeaconEscrow.sol	ee088e3de3fe90cef692d4ecbc71a0973d54cca287ab9d6ca53f34df0df38f80
● UBB	sto/UpgradeableBeacon/UpgradeableBeaconToken.sol	e59b1c1fb2a7a6bd11cdbbc1c9a468c285899ca0dbd46ee430c66d13b2997c85
● STP	sto/UpgradeableTemplate/STOEscrowUpgradeable.sol	87f0a0fb79755d218caab0bbc3351930f415658bb745ce2d1700f19d3550a83b
● SOT	sto/UpgradeableTemplate/STOTokenCheckpointsUpgradeable.sol	ce6e77c1a2c220a4e041cb9aaebe577f98ef6af76b6920883a8f5a26135409c7
● SOC	sto/UpgradeableTemplate/STOTokenConfiscateUpgradeable.sol	8bb0f295e28efb2e6255ba5c1c02b25e242607d63d0f3af495ef76243f9f1ba1
● SOD	sto/UpgradeableTemplate/STOTokenDividendUpgradeable.sol	0f4a89280231641a472b4795b306f6390c989703000c446659db850e209e92c3
● SOU	sto/UpgradeableTemplate/STOTokenUpgradeable.sol	a1cf88f2d513dfbf41cd819fd4ded25e29300fa374dcd739b291fcf0f0350890
● STF	sto/STOFactory.sol	f04e9191bf68de2b6b0fc8737148e1086f5e774e30b2b683216370805def8de4
● SOB	sto/helpers/STOBeaconProxy.sol	5e2382355ea67c10491373d761399fe101d425228cfbee6f9f6fbd8428b8ae7a
● SOE	sto/helpers/STOErrors.sol	d09dcc53f995a2150288423381117196fc72240704eba1c99188a684f512acb3
● IRK	sto/interfaces/IRouter.sol	af3cd17792059d3c1d38bbaa2a9aabf3078b71deb756e011eb76fac98b4a3e41
● ISE	sto/interfaces/ISTOEscrow.sol	a367f0068c9663a8bf9bf89fe32069041f9364fed3a3abacdd0e2c1f11009d88
● ISC	sto/interfaces/ISTOToken.sol	7b6db99dc17a3a5206526bd21a521b66618bb6a1c7c48f3aaf15ab4da9643074

APPROACH & METHODS | BRICKKEN - AUDIT

This report has been prepared for Brickken to discover issues and vulnerabilities in the source code of the Brickken - Audit project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Manual Review and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

FINDINGS | BRICKKEN - AUDIT

15	1	3	2	4	5
Total Findings	Critical	Major	Medium	Minor	Informational

This report has been prepared to discover issues and vulnerabilities for Brickken - Audit. Through this audit, we have uncovered 15 issues ranging from different severity levels. Utilizing the techniques of Manual Review & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
SOB-01	Missing Access Control For Beacon Upgrade	Logical Issue	Critical	Resolved
CKP-01	**Centralized Control Of Contract Upgrade**	**Centralization**	**Major**	**Acknowledged**
CKP-02	**Centralization Related Risks**	**Centralization**	**Major**	**Acknowledged**
STP-01	Potential Flashloan Attack	Logical Issue	Major	Acknowledged
SOU-01	`_afterTokenTransfer()` Incorrectly Updating Information Of Holders	Logical Issue	Medium	Resolved
STP-02	Raised Amount Not Properly Calculated	Logical Issue	Medium	Resolved
CKP-03	Missing Zero Address Validation	Volatile Code	Minor	Resolved
STP-03	Not All Parent Contract Initializing Functions Are Called	language-specific	Minor	Resolved
STP-04	Third Party Dependency	Volatile Code	Minor	Acknowledged
STP-05	`setRouter()` Function Does Not Remove Allowance	Logical Issue	Minor	Resolved
CKP-04	Unlocked Compiler Version	language-specific	Informational	Resolved

ID	Title	Category	Severity	Status
CKP-05	Typos In The Contracts	Coding Style	Informational	Resolved
STP-06	Usage Of Magic Numbers	Coding Style	Informational	Resolved
STP-07	Finishing STO Before `endDate` When `hardcap` Is Reached	Logical Issue	Informational	Resolved
UTP-01	Missing Emit Events	Coding Style	Informational	Resolved

SOB-01 | MISSING ACCESS CONTROL FOR BEACON UPGRADE

Category	Severity	Location	Status
Logical Issue	Critical	sto/helpers/STOBeaconProxy.sol (V2): 78~80	Resolved

Description

The linked function allows any account to update the beacon contract address of the proxy, which means anyone can upgrade the implementation contract behind this proxy.

```
78    function upgradeBeaconToAndCall(address beacon, bytes memory data) public  {
79          _upgradeBeaconToAndCall(beacon, data, false);
80      }
```

Recommendation

We recommend enforcing only the `admin` role can access the linked function.

Alleviation

The unprotected function was deleted in the commit hash: https://github.com/Brickken/brickken-protocol/commit/3bab05fd798354ca0fdf3cbfb5b79bcfae456727

CKP-01 | CENTRALIZED CONTROL OF CONTRACT UPGRADE

Category	Severity	Location	Status
Centralization	Major	sto/STOFactory.sol (V2): 13; sto/UpgradeableBeacon/UpgradeableBeaconEscrow.sol (V2): 19; sto/UpgradeableBeacon/UpgradeableBeaconToken.sol (V2): 18; sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 9; sto/UpgradeableTemplate/STOTokenCheckpointsUpgradeable.sol (V2): 17; sto/UpgradeableTemplate/STOTokenConfiscateUpgradeable.sol (V2): 8; sto/UpgradeableTemplate/STOTokenDividendUpgradeable.sol (V2): 9; sto/UpgradeableTemplate/STOTokenUpgradeable.sol (V2): 8	Acknowledged

Description

The highlighted contracts are upgradeable contracts, the owner can upgrade them without the community's commitment. If an attacker compromises the account, they can change the implementation of the contracts and drain tokens from them.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Brickken Team] Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

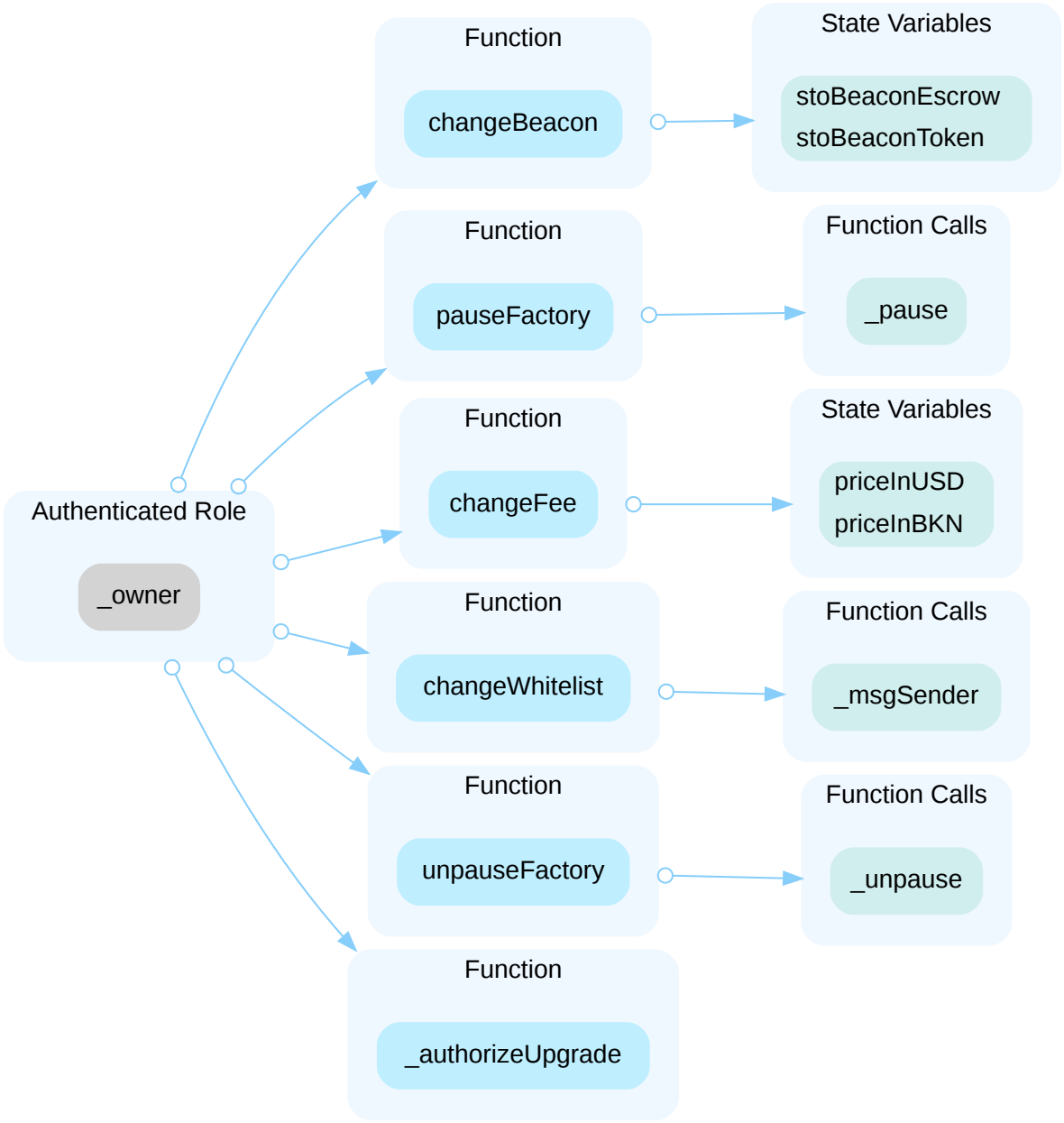
We will include a DAO managed by BKN holders to decide adjustable parameters of the STO Factory. It's in our roadmap and we leave it for either 2023 or 2024.

CKP-02 | CENTRALIZATION RELATED RISKS

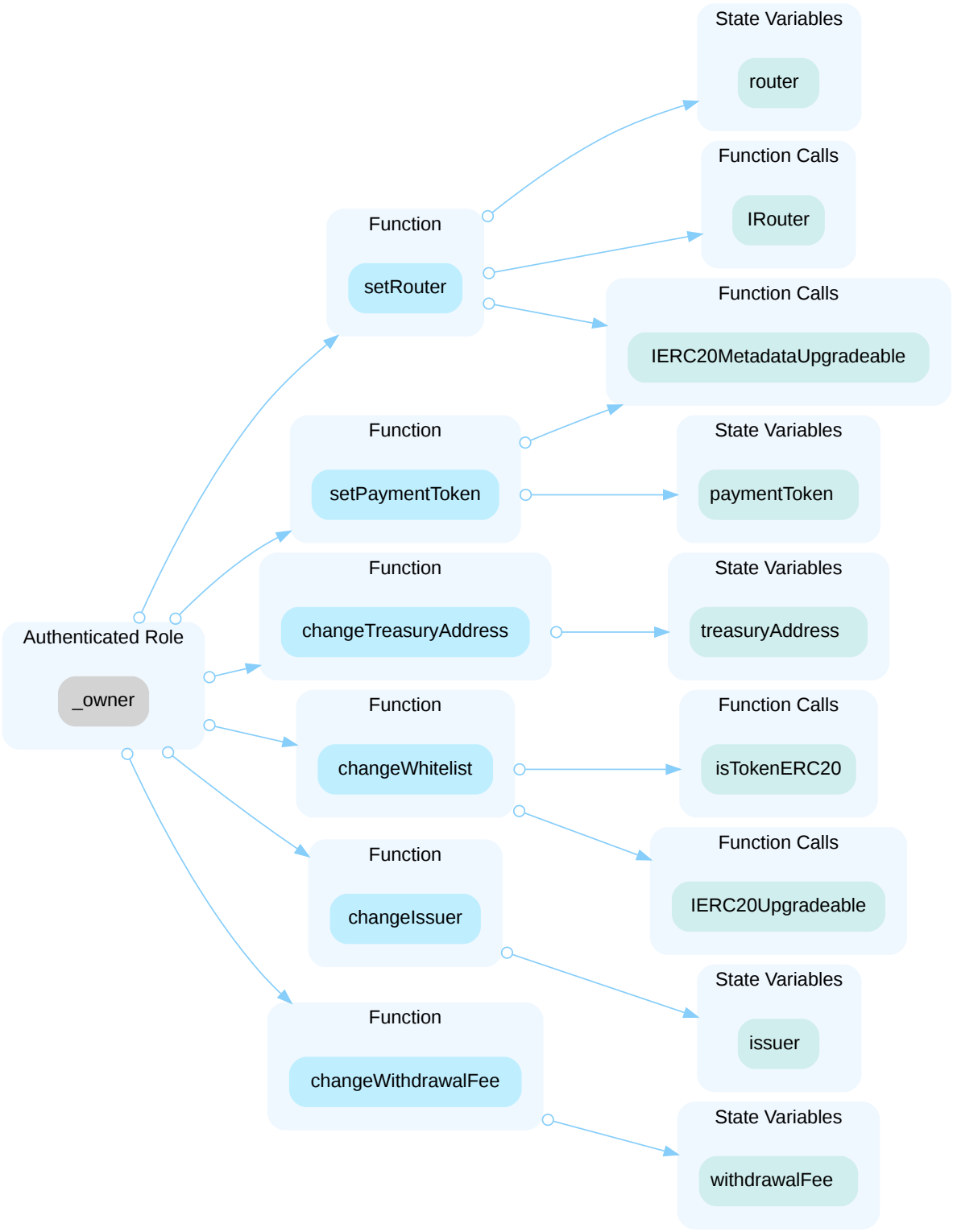
Category	Severity	Location	Status
Centralization	Major	sto/STOFactory.sol (V2): 137, 142, 149, 166, 183, 288; sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 243, 252, 264, 291, 333, 343; sto/UpgradeableTemplate/STOTokenConfiscateUpgradeable.sol (V2): 32, 46, 53; sto/UpgradeableTemplate/STOTokenDividendUpgradeable.sol (V2): 135; sto/UpgradeableTemplate/STOTokenUpgradeable.sol (V2): 94, 113, 121, 129	Acknowledged

Description

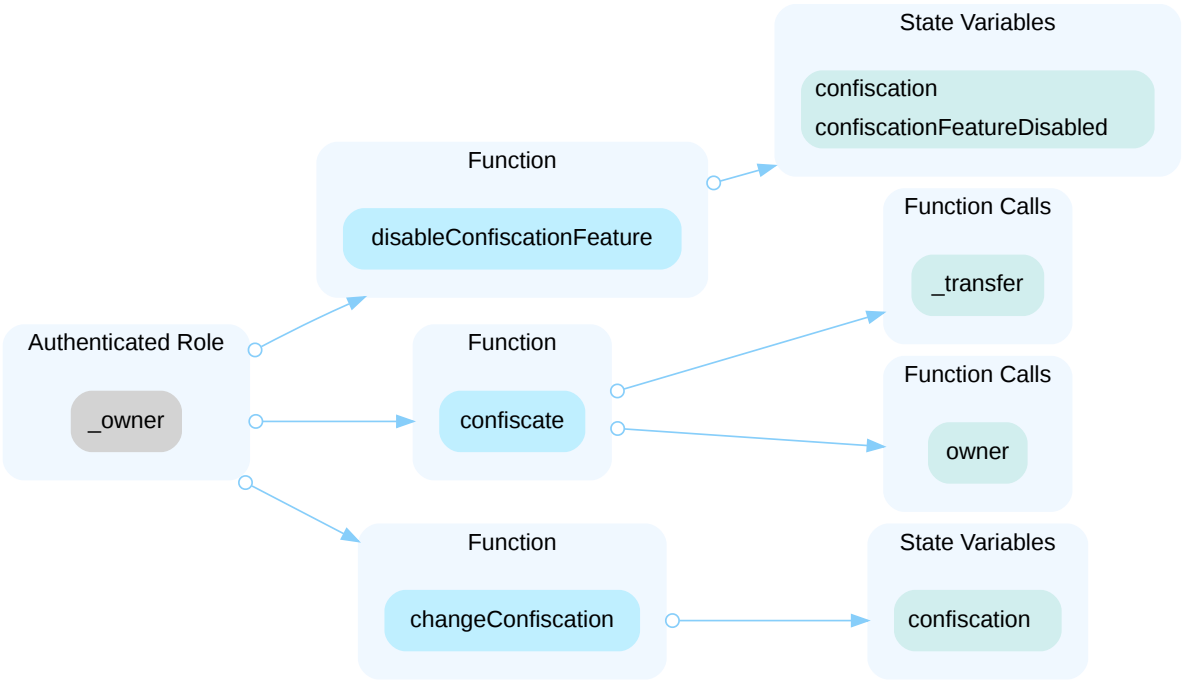
In the contract `STOFactory` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and change the fees.



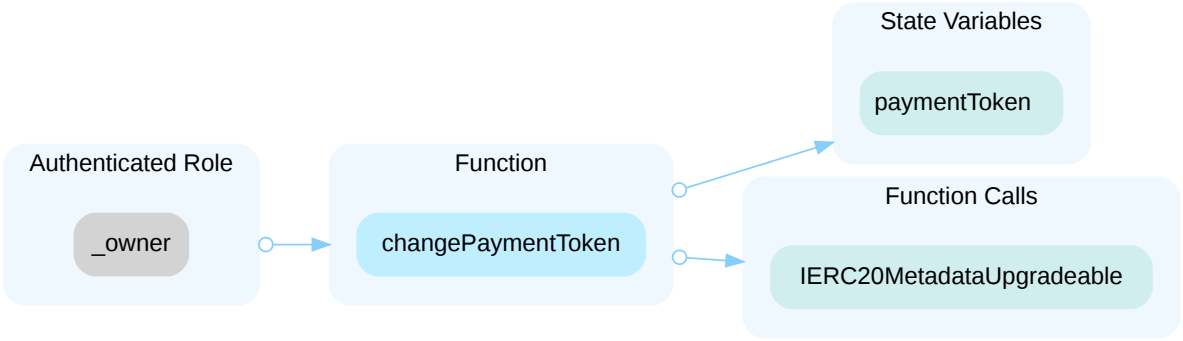
In the contract `STOEscrowUpgradeable` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and change the escrow's parameters, such as the treasury address, the router, and the withdrawal fee.



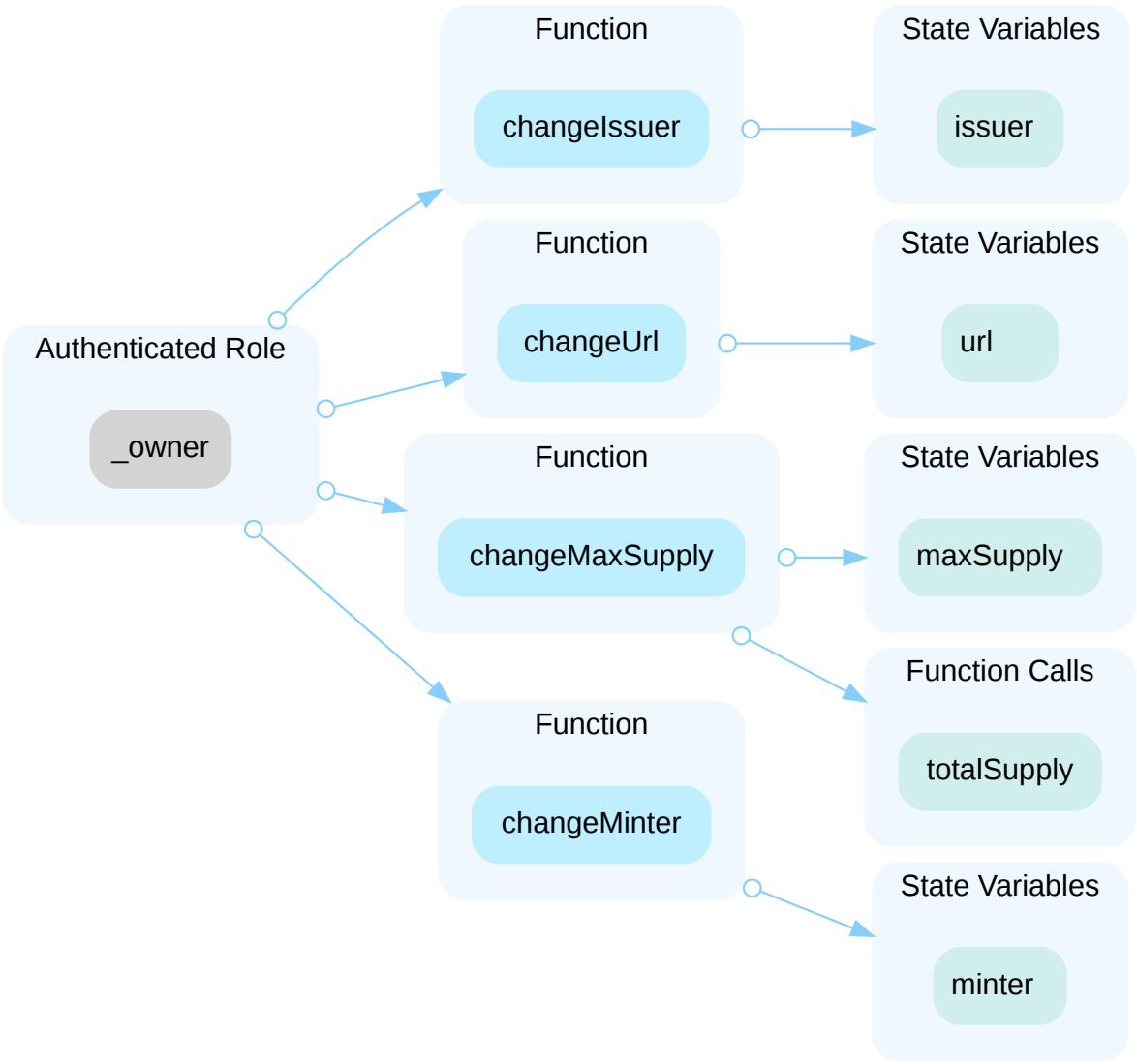
In the contract `STOTokenConfiscateUpgradeable` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and steal user tokens.



In the contract `STOTokenDividendUpgradeable` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and change the payment token.



In the contract `STOTokenUpgradeable` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and change the issuer and minter roles, change the IPFS URI where all documents of the tokenization are stored, and change the maximum supply of `STOtoken`.



Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND

- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Brickken Team] Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

As of right now, given the legal implications of running Security Token Offerings, the tokenizer must have some powers in order to ensure that no legal clauses are broken within their STOs.

The tokenizer can always renounce ownership, freezing the configuration of their Escrow and Token, and some functionalities can be disabled forever like the confiscation.

Contracts are upgradeable but once ownership is renounced, the protocol will be lock forever and the mentioned risks will be removed.

STP-01 | POTENTIAL FLASHLOAN ATTACK

Category	Severity	Location	Status
Logical Issue	Major	sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 446, 609, 789	Acknowledged

Description

Flash loans are a way to borrow large amounts of money for a certain fee. The requirement is that the loans need to be returned within the same transaction in a block. If not, the transaction will be reverted.

An attacker can use the borrowed money as the initial funds for an exploit to enlarge the profit and/or manipulate the token price in the decentralized exchanges.

During the audit, it was found that the `buyToken()` and `getPriceInPaymentToken()` functions rely on price calculations that are based on-chain (`router.getAmountsOut()`), meaning that they would be susceptible to flash-loan attacks by manipulating the price of given pairs to the attacker's benefit.

Recommendation

If the project requires price references, caution should be taken to avoid flash loan attacks involving price manipulation. To minimize the chance of this happening, we recommend:

1. using multiple reliable on-chain price oracle sources, such as Chainlink or Band protocol.
2. using the Time-Weighted Average Price (TWAP). The TWAP represents the average price of a token over a specified time frame. If an attacker manipulates the price in one block, it will not affect the average price as drastically.
3. forcing critical transactions to span at least two blocks, if the contract use cases allow it. Flash loans only allow users to borrow money within a single transaction.

Alleviation

The Brickken team indicated the following: "We will use TWAP oracles with a safe window period. The first version of the protocol doesn't require Uniswap at all since the only form of payment will be the `paymentToken` defined in the Escrow, and this will not require Uniswap to be used initially."

SOU-01 `_afterTokenTransfer()` INCORRECTLY UPDATING INFORMATION OF HOLDERS

Category	Severity	Location	Status
Logical Issue	Medium	sto/UpgradeableTemplate/STOTokenUpgradeable.sol (V2): 148~166	Resolved

Description

```
157    uint256 balanceFrom = balanceOf(from);
158
159    // If it's minting, the from is address(0) so no need to account for that
160
// If it's burning, the balanceFrom will account if now the original user has no
tokens

161    if(from != address(0) && balanceFrom == 0) {
162      totalHolders = totalHolders - 1;
163      holders[from] = false;
164    }
165
166    // If it's minting, the `to` is checked to already exist as holder or not
167    // If it's burning, the `to` is the zero address so no need to account for it
168    if(amount > 0 && !holders[to] && to != address(0)) {
169      totalHolders = totalHolders + 1;
170      holders[from] = true;
171    }
```

Given the first `if` statement above, because ERC20 allows transfers of 0 tokens, an attacker without any token could call `transfer(0)` to incorrectly decrement the value of `totalHolders`. If `totalHolders` is reduced to 0, future transfers that would empty the caller's balance would revert. This could potentially cause a denial of service in certain scenarios where holders want to burn or sell all tokens.

Furthermore, for the second `if` statement(L166) is not properly updating the new holders. The `holders[from] = true` should actually be `holders[to] = true`.

The two logical errors indicated above could make the whole tracking of holders inconsistent.

Recommendation

We recommend the development team to review the `_afterTokenTransfer()` internal function.

Alleviation

Changes have been reflected in the commit hash: https://github.com/Brickken/brickken-protocol/commit/55e9e075f9984f08ef4ba7233676e23c456990cf

STP-02 | RAISED AMOUNT NOT PROPERLY CALCULATED

Category	Severity	Location	Status
Logical Issue	Medium	sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 552~556	Resolved

Description

The highlighted lines are not correctly calculating the raised amount per issuance.

Notice that the code snippet is dividing the amount of USD that has been paid (`actualAmount` variable) with the price of the STO token in USD (`issuances[issuanceIndex].priceInUSD`). This gives the amount of STO tokens bought and not the total USD that has been raised so far.

Impact

Issuances will hold an incorrect value on the `raisedAmount` element. This would contain the number of STO tokens bought instead of the actual amount of USD that was raised.

Please note that the reasoning behind this is that every `paymentToken` will be a stablecoin (USDC/USDT/DAI) so that tokenizers do not lose money on volatility while running STOs. In this way it is possible to calculate the amount in USD (to the 18 decimals) and not the amount of STO tokens bought.

Recommendation

It is recommended to review the calculation of the `raisedAmount` value in the `buyToken()` function.

Alleviation

The issue was fixed in commit: https://github.com/Brickken/brickken-protocol/pull/16/commits/e5cdd0f3d181ab3ceed747bc724880d62b7d7280.

CKP-03 | MISSING ZERO ADDRESS VALIDATION

Category	Severity	Location	Status
Volatile Code	Minor	sto/STOFactory.sol (V2): 130; sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 236~237; sto/UpgradeableTemplate/STOTokenUpgradeable.sol (V2): 73	Resolved

Description

Addresses should be checked before assignment or external call to make sure they are not zero addresses.

Recommendation

We advise adding a zero-check for the passed-in address value to prevent unexpected errors.

Alleviation

Changes have been reflected in the commit hash: https://github.com/Brickken/brickken-protocol/commit/79161d71b23618a19c0af30cb63a66535afd7ea6 and https://github.com/Brickken/brickken-protocol/commit/aeb5ad9e5498160b706eccd472589b035f033724

STP-03 NOT ALL PARENT CONTRACT INITIALIZING FUNCTIONS ARE CALLED

Category	Severity	Location	Status
language-specific	Minor	sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 213~239	Resolved

Description

Contract `STOEscrowUpgradeable` extends `OwnableUpgradeable.sol`, while `__Ownable_init_unchained()` is not called in the initialize function. Generally, the initializer function of an upgradeable contract should always call all the initializer functions of the contracts that it extends.

Recommendation

We advise the client to call `__Ownable_init_unchained()` in the linked contract.

Alleviation

Changes have been reflected in the commit hash: https://github.com/Brickken/brickken-protocol/commit/ff55e45e1ac53a8f24c7866c3a12ccb3576e4a25

STP-04 | THIRD PARTY DEPENDENCY

Category	Severity	Location	Status
Volatile Code	Minor	sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 5, 77 ~78, 235	Acknowledged

Description

The contract is serving as the underlying entity to interact with one or more third party protocols. The scope of the audit treats third party entities as black boxes and assume their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. In addition, upgrades of third parties can possibly create severe impacts, such as increasing fees of third parties, migrating to new LP pools, etc.

Recommendation

We understand that the business logic requires interaction with the third parties. We encourage the team to constantly monitor the statuses of third parties to mitigate the side effects when unexpected activities are observed.

Alleviation

The Brickken team stated: "We will setup on-chain monitoring solutions such as Forta detection bots to monitor all instances of Factories, Escrows and Tokens across all possible networks. We do have already templates in place and testing them out."

STP-05 | `setRouter()` FUNCTION DOES NOT REMOVE ALLOWANCE

Category	Severity	Location	Status
Logical Issue	Minor	sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 264	Resolved

Description

When setting a new router with the `setRouter()` function, the allowance previously given to the now-replaced router is not removed.

Recommendation

We recommend removing the allowance previously given to the replaced router.

Alleviation

Changes have been reflected in the commit hash: https://github.com/Brickken/brickken-protocol/commit/dbe975379035ff778b3c11f2b8246f56f6266ea5

CKP-04 | UNLOCKED COMPILER VERSION

Category	Severity	Location	Status
language-specific	Informational	sto/STOFactory.sol (V2): 2; sto/UpgradeableBeacon/UpgradeableBeaconEscrow.sol (V2): 3; sto/UpgradeableBeacon/UpgradeableBeaconToken.sol (V2): 2; sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 2; sto/UpgradeableTemplate/STOTokenCheckpointsUpgradeable.sol (V2): 2; sto/UpgradeableTemplate/STOTokenConfiscateUpgradeable.sol (V2): 2; sto/UpgradeableTemplate/STOTokenDividendUpgradeable.sol (V2): 2; sto/UpgradeableTemplate/STOTokenUpgradeable.sol (V2): 2; sto/helpers/STOBeaconProxy.sol (V2): 2; sto/helpers/STOErrors.sol (V2): 2; sto/interfaces/IRouter.sol (V2): 2; sto/interfaces/ISTOEscrow.sol (V2): 2; sto/interfaces/ISTOToken.sol (V2): 2	Resolved

Description

The contract has unlocked compiler version. An unlocked compiler version in the source code of the contract permits the user to compile it at or above a particular version. This, in turn, leads to differences in the generated bytecode between compilations due to different compiler versions. This can lead to an ambiguity when debugging as compiler specific bugs may occur in the codebase that would be hard to identify over a span of multiple compiler versions rather than a specific one.

Recommendation

We advise that the compiler version is instead locked at the lowest version possible that the contract can be compiled at. For example, for version `v0.8.17` the contract should contain the following line:

```
pragma solidity 0.8.17;
```

Alleviation

Changes have been reflected in the commit hash: https://github.com/Brickken/brickken-protocol/commit/021b6d0987015fca55555d543e167af9a2e876ef

CKP-05 | TYPOS IN THE CONTRACTS

Category	Severity	Location	Status
Coding Style	Informational	sto/STOFactory.sol (V2): 66, 92, 94, 98, 163; sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 19, 20, 60, 61, 107, 748, 928, 934, 936, 936, 954, 960; sto/UpgradeableTemplate/STOTokenCheckpointsUpgradeable.sol (V2): 10; sto/UpgradeableTemplate/STOTokenUpgradeable.sol (V2): 18, 97; sto/helpers/STOErrors.sol (V2): 29, 29, 88; sto/interfaces/ISTOEscrow.sol (V2): 11, 12, 26, 52, 174, 208, 210, 236; sto/interfaces/ISTOToken.sol (V2): 4, 17	Resolved

Description

There are a few typos in the contracts highlighted in the `Locations` section.

Recommendation

We recommend correcting all typos in the contract.

Alleviation

Changes have been reflected in the commit hash: https://github.com/Brickken/brickken-protocol/commit/8cbafe2e8246157f0ce126a493d2eb4aad344802

STP-06 | USAGE OF MAGIC NUMBERS

Category	Severity	Location	Status
Coding Style	Informational	sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 334, 947	Resolved

Description

The maximum withdrawal fee allowed was checked using a hardcoded number instead of an explicit constant. Literals with many digits are difficult to read and review.

Recommendation

We recommend declaring constants to improve code maintainability and readability. Furthermore, we recommend using scientific notation (e.g. `1e4`) or underscores (e.g. `10_000`) to improve readability.

Alleviation

Changes have been reflected in the commit hash: https://github.com/Brickken/brickken-protocol/commit/228e1df135fa28210e3e96c5649b9fa18c3f7c8a

STP-07 FINISHING STO BEFORE `endDate` WHEN `hardcap` IS REACHED

Category	Severity	Location	Status
Logical Issue	Informational	sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 356 ~357	Resolved

Description

Since commit https://github.com/Brickken/brickken-protocol/pull/16/commits/a1fa6c39e254c98138c9ab0bcbde3b3082c09947, `STOEscrowUpgradeable` allows issuances to be finished before its `endDate` if the `hardcap` is reached.

In the previous version, even if an STO had achieved its `hardcap`, it could only be finished if `endDate` was also reached.

Recommendation

The past design did not pose any security threat, but if the desired behaviour is that a tokeniser should be able to close the STO ahead of time if the `hardcap` is achieved, changes in the design had to be made as observed in the mentioned commit.

UTP-01 | MISSING EMIT EVENTS

Category	Severity	Location	Status
Coding Style	Informational	sto/UpgradeableTemplate/STOTokenConfiscateUpgradeable.sol (V2): 32, 46, 53; sto/UpgradeableTemplate/STOTokenUpgradeable.sol (V2): 56	Resolved

Description

There should always be events emitted in the sensitive functions that are controlled by centralization roles.

Recommendation

It is recommended emitting events for the sensitive functions that are controlled by centralization roles.

Alleviation

Changes have been reflected in the commit hash: https://github.com/Brickken/brickken-protocol/commit/2c95f6dac7c6031f9856dd084223713584b9c933

OPTIMIZATIONS | BRICKKEN - AUDIT

ID	Title	Category	Severity	Status
STP-08	Costly Operations Inside A Loop	Gas Optimization	Optimization	Acknowledged

STP-08 | COSTLY OPERATIONS INSIDE A LOOP

Category	Severity	Location	Status
Gas Optimization	Optimization	sto/UpgradeableTemplate/STOEscrowUpgradeable.sol (V2): 291~329	Acknowledged

Description

The computation for each iteration is complex, it could lead to gas insufficiency when the length of the array `tokensToChange` is large.

```
function changeWhitelist(
        address[] calldata tokensToChange,
        uint256[] calldata multipliers,
        bool[] calldata statuses
    ) external onlyOwner {
        for (uint256 i = 0; i < tokensToChange.length; i++) {
```

Recommendation

We recommend setting appropriate constraints on the length of the array to avoid potential gas exhaustion.

Alleviation

The Brickken team stated: "the function is an `onlyOwner` function so we will manage how many elements are used. The fact that is unbounded is just to leave open the possibility of setting things in batch, but it's not mandatory to do so."

APPENDIX | BRICKKEN - AUDIT

Finding Categories

Categories	Description
Centralization	Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.
Gas Optimization	Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.
Logical Issue	Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion on how block.timestamp works.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases that may result in a vulnerability.
	Language Specific findings are issues that would only arise within Solidity, i.e. incorrect usage of private or delete.
Coding Style	Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

CertiK | Securing the Web3 World

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.